FlexSight

Temperature Monitoring & Alert System

Temperature Sensor -> ESP32 Monitoring Device -> MQTT/API -> Server -> Database -> Web Dashboard -> Dashboard Alert / Email Notification

Final Report - Filtered Version

This report documents the formation, ideation, and design of FlexSight as a lightweight IoT-based system for hourly temperature monitoring, dashboard alerts, and email notifications.

Current Scope	Reading Frequency	Core Users	Alert Thresholds
Temperature Monitoring Only	Hourly	Owner / Admin / Inspector	Warning 45-49C | Critical 50C+

1. Team Formation & Roles

The team was structured around the main areas required to build FlexSight: UI/UX design, frontend dashboard development, backend development, ESP32 device setup, API communication, database integration, and alert processing.

Team Member	Role	Responsibility
Hamsa Bnian Alammar	Team Leader & Frontend Developer	Leads team coordination, follows up on tasks, supports planning, and develops frontend dashboard pages using HTML, CSS, and JavaScript.
Munirah Enad Alotaibi	UI/UX Designer & Frontend Developer	Designs Figma wireframes, dashboard layout, visual identity, reusable components, and supports frontend implementation based on the final UI design.
Hanin Taqi Al Sayed Hassan	Programming & Backend Developer	Handles programming tasks, ESP32 monitoring device setup, temperature sensor integration, MQTT/API communication, and supports alert processing.
Rabea Younis Thabit	Backend Developer	Builds and supports the backend structure, server-side processing, API handling, database connection, and temperature threshold checking.

2. Collaboration Strategy

Communication Tools

WhatsApp for quick updates, Discord for technical discussions, Google Meet for remote meetings, and in-person sessions for brainstorming and planning.

Documentation & Design

Google Docs for collaborative writing, Canva for visual materials, GitHub for version control, and Figma for UI/UX design and prototype screens.

Decision-Making

Decisions are made through group discussion. If no clear agreement is reached, voting is used based on feasibility, clarity, technical alignment, project timeline, and MVP simplicity.

The collaboration approach supports clear communication, organized documentation, design consistency, and controlled version management during the MVP development process.

3. Brainstorming & Idea Evaluation

The team explored several ideas related to safety response and monitoring. The selected idea needed to be feasible, simple, technically aligned, and suitable for a focused MVP.

Mind Mapping

Organized system components, communication methods, sensor options, and alert features visually.

How Might We Questions

Explored ways to detect abnormal temperature levels and send alerts efficiently.

Research-Based Discussions

Focused on practical safety challenges, delayed detection, and dashboard-based monitoring.

SCAMPER

Helped improve the idea by modifying existing temperature monitoring concepts to fit the MVP scope.

4. Evaluation Criteria

The ideas were compared using five criteria. FlexSight scored highest because it balances project feasibility with practical value.

Feasibility

Can be built within the project timeline and available resources.

Impact

Improves response to abnormal temperature levels and supports safer operations.

Technical Alignment

Fits the team skills in MQTT/API, backend, database, and dashboard development.

Scalability

Can support more ESP32 monitoring devices and future dashboard improvements.

MVP Simplicity

Keeps the project focused on temperature monitoring without unnecessary complexity.

5. Ideas Explored & Rejected

Idea	Description	Result
Advanced AI Vision System	Vision-based system for detecting operational risks using AI.	Rejected - too complex for the current MVP.
Full Fire Detection System	Multi-sensor safety detection system.	Rejected - requires more hardware testing and exceeds the project timeline.
Mobile Alert Application	Mobile app for receiving alerts.	Rejected - not essential because dashboard alerts and email notifications cover the core need.
FlexSight	Temperature monitoring system using ESP32 devices, MQTT/API communication, backend processing, dashboard alerts, and email notifications.	Selected.

6. Evaluation Rubric / Scoring

Idea	Feasibility	Impact	Technical Alignment	Scalability	MVP Simplicity	Total
Advanced AI Vision System	2	5	3	5	2	17
Full Fire Detection System	3	5	4	5	2	19
Mobile Alert Application	4	3	4	3	4	18
FlexSight	5	4	5	5	5	24

Risk Summary

Advanced concepts were rejected because they exceeded the current MVP timeline and team capacity. FlexSight was selected because it provides a focused temperature monitoring workflow that can be implemented and demonstrated within the project scope.

7. Final Decision & Project Overview

Based on the evaluation results, the team selected FlexSight as the final MVP because it is feasible, simple to implement, aligned with team skills, and scalable for future development.

Temperature Sensor -> ESP32 Monitoring Device -> MQTT/API -> Server -> Database -> Web Dashboard -> Dashboard Alert / Email Notification

FlexSight is a web-based temperature monitoring and alert system. The ESP32 monitoring device collects hourly readings, the backend validates and stores the readings, and the dashboard displays device status, temperature readings, and alerts.

8. Expected Outcomes

- Hourly temperature monitoring using ESP32 monitoring devices.
- Dashboard visibility of temperature readings and device status.
- Warning and critical alerts based on temperature thresholds.
- Email notifications when abnormal temperature readings are detected.
- Stored historical readings for review.
- Clear user access based on Owner, Admin, and Inspector roles.

Hourly Monitoring

The ESP32 monitoring device collects one temperature reading every hour.

Centralized Processing

The backend validates readings, checks thresholds, stores records, and prepares dashboard data.

Alerts & Email

Dashboard alerts and email notifications are generated when warning or critical thresholds are exceeded.

9. Problem Statement & Proposed Solution

The Problem	Proposed Solution
Manual checking can delay response to abnormal temperature levels. Readings may not be stored centrally, and responsible users may not be informed quickly when a temperature issue occurs.	FlexSight collects hourly temperature readings through ESP32 devices, sends them through MQTT/API, processes them on the backend, stores them in a SQL database, displays them on a web dashboard, and sends email notifications when alerts are generated.

Threshold Rules

Status	Temperature Range	System Response
Normal	Below 45C	Store the hourly reading and display normal status.
Warning	45C to 49C	Create warning alert and notify responsible users.
Critical	50C or above	Create critical alert, display it on the dashboard, and send email notification.

10. User Roles

FlexSight uses role-based access control to keep each user focused on the actions needed for the MVP.

Role	Description
Owner	Full access to users, devices, readings, alerts, threshold settings, and system settings.
Admin	Monitors temperature readings, device status, alerts, and dashboard activity.
Inspector	Follows up on assigned alerts, reviews affected devices, adds notes, and updates incident status.

Role Scope

- The MVP includes only Owner, Admin, and Inspector roles.
- The Admin role is used as a standalone role.
- Access is based on the three final roles: Owner, Admin, and Inspector.

11. Technical Stack

Layer	Technology	Role
Sensor Layer	Temperature Sensor	Collects temperature readings.
Device Layer	ESP32 Monitoring Device	Collects one temperature reading every hour and sends it to the backend.
Communication Layer	MQTT/API	Sends hourly temperature readings from the ESP32 device to the server.
Server Layer	Python / Flask Backend	Receives readings, validates data, checks thresholds, and processes alerts.
Database Layer	SQL Database	Stores users, devices, temperature readings, alerts, threshold settings, and email notification records.
Dashboard Layer	HTML, CSS, JavaScript	Displays hourly readings, device status, alert summaries, and system information.
Design Layer	Figma	Supports wireframes, UI design, dashboard screens, and prototype.
Version Control	GitHub	Supports repository hosting, version control, and team collaboration.

12. Strengths, Weaknesses & Constraints

Strengths	Constraints / Risks	Mitigation
Simple and achievable MVP	Internet connectivity may affect data transmission.	Test connection stability and handle failed transmission attempts.
Focused on temperature monitoring only	Sensor calibration affects accuracy.	Calibrate the temperature sensor before testing.
Dashboard alerts and email notifications	Hardware quality may affect readings.	Use approved and tested components.
Structured storage in SQL database	One threshold may not fit every use case.	Allow warning and critical thresholds to be updated in settings.
Clear role-based access	Email delivery may fail due to service configuration.	Test email delivery and track notification status.

13. Added Value & Future Enhancements

For Responsible Users

Clear dashboard visibility, easier monitoring of device status, faster response to warning and critical alerts, and access to historical hourly readings.

For Development Team

Practical experience with IoT communication, backend processing, database design, alert logic, and dashboard implementation.

For Future Development

The MVP structure can support more ESP32 devices, configurable thresholds, alert assignment, email delivery tracking, and improved dashboard summaries.

Future Feature	Description
More ESP32 Devices	Support additional ESP32 monitoring devices.
Configurable Thresholds	Allow warning and critical thresholds to be customized per device.
Alert Assignment	Assign alerts to specific responsible users.
Email Delivery Tracking	Track whether alert emails were sent successfully.
Advanced Export	Export temperature readings and alert history.
Dashboard Summaries	Show summaries for readings, alerts, and device status.

14. Future Expansion & Decision Justification

The team selected FlexSight because it addresses a clear temperature monitoring need, can be built using simple and available components, and fits the project timeline and team capabilities.

Core workflow: Temperature Sensor -> ESP32 Monitoring Device -> MQTT/API -> Server -> Database -> Web Dashboard -> Dashboard Alert / Email Notification

Project Summary

FlexSight collects hourly temperature readings from ESP32 monitoring devices, sends the readings to the backend through MQTT/API, validates and stores the data, checks warning and critical thresholds, and displays device status, readings, and alerts on the web dashboard.

The MVP is limited to temperature monitoring only. Mobile apps, external messaging channels, additional sensor categories, advanced prediction features, power monitoring, HVAC monitoring, energy monitoring, advanced analytics, and third-party integrations are intentionally kept out of scope for the current version.

Final Scope	Final Roles	Frequency	Notifications
Temperature monitoring only	Owner / Admin / Inspector	Hourly readings	Dashboard alert / Email notification